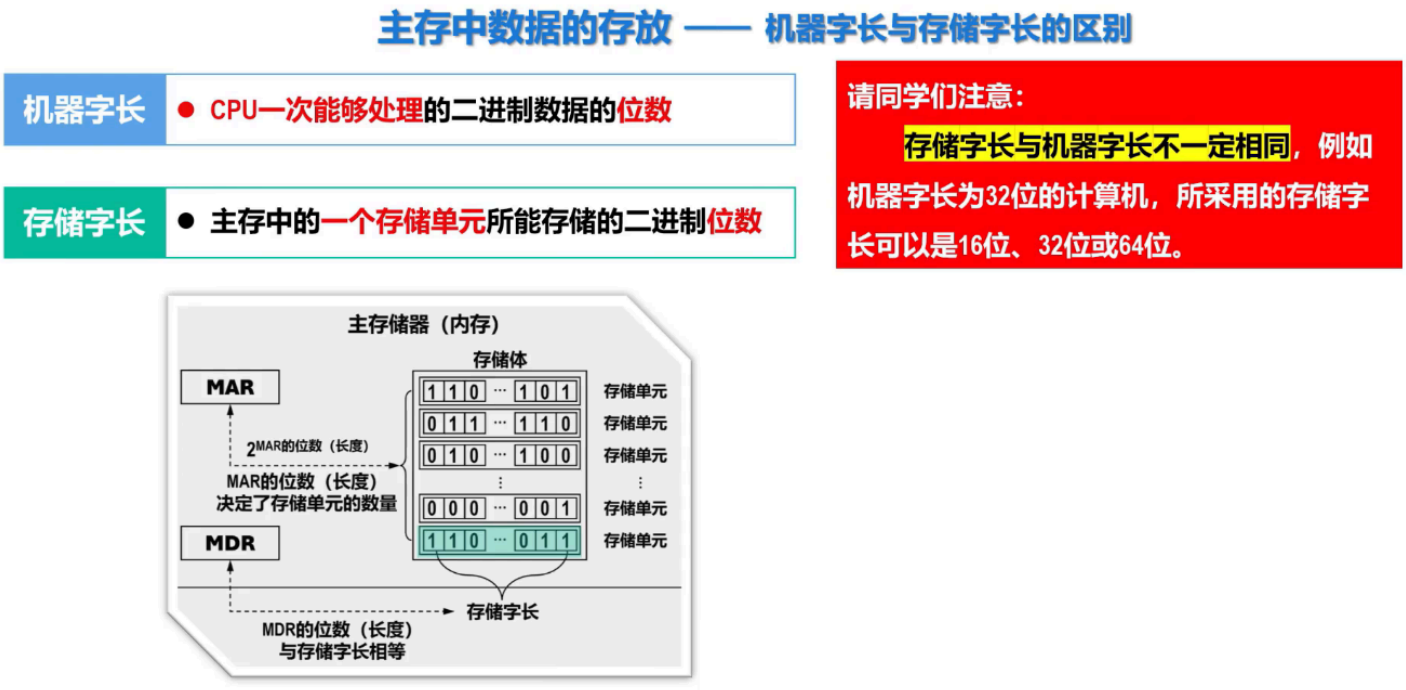
4.5A 主存的数据存放

考纲定位：计算机组成原理 · 第 4 章 存储器（4.5A） | 重要度 ★★★★ | 常见考法：选择题（机器字长/存储字长、字节/半字/字地址、大端小端、边界对齐）

一句话核心：机器字长是 CPU 一次处理的位数，存储字长是一个存储单元的位数；多字节数据要同时考虑“字节顺序（大小端）”和“地址对齐”

一、机器字长与存储字长



概念	定义	决定什么
机器字长	CPU 一次能处理的二进制位数	寄存器、ALU、数据总线宽度
存储字长	一个存储单元存放的二进制位数	MDR 位数、一次访存取出的位数
指令字长	一条指令的二进制位数	PC 增量、取指次数

提炼

- 三者可以相等，也可以不等；题目给出“32 位计算机”只说明机器字长，不代表存储字长
- 若存储字长 16 位、机器字长 32 位，则一次处理 32 位要访问两个存储单元

易错

- 机器字长 ≠ 存储字长 ≠ 指令字长
- MDR 位数 = 存储字长；MAR 位数决定存储单元数

二、地址访问模式

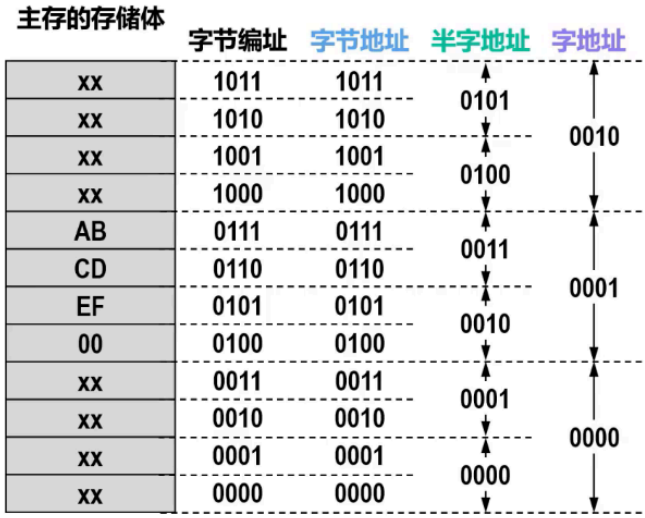
主存中数据的存放 —— 地址访问模式

■ 主存通常按字节进行编址，而存储字长（主存中的一个存储单元所能存储的二进制位数）是字节的2的整数次幂倍（例如1字节，2字节，4字节）。

■ 以机器字长为32位的计算机为例，对主存的访问既可以按字节访问，也可以按16位半字访问，还可以按32位字访问。因此，可将主存地址分为：

- ☐ 字节地址
- ☐ 半字地址
- ☐ 字地址

【举例】



提炼

- 主存按字节编址时，每个字节一个地址；半字地址、字地址由字节地址右移得到
- 字节地址低 k 位用于选择“字内哪个字节”，高位用于选择“第几个字”
- 若字长 4B，则
  - 字节地址 = 字地址 × 4 + 字内偏移（2 位）
  - 半字地址 = 字地址 × 2 + 半字内偏移（1 位）
  - 字地址 = 字节地址 >> 2

公式

$$\text{字地址} = \lfloor \text{字节地址} / \text{每字字节数} \rfloor$$

举例

主存中数据的存放 —— 地址访问模式

【举例】

主存的存储体

	字节编址	字节地址	半字地址	字地址
xx	1011	1011	0101	0010
xx	1010	1010		
xx	1001	1001	0100	
xx	1000	1000		
AB	0111	0111	0011	0001
CD	0110	0110		
EF	0101	0101	0010	
00	0100	0100		
xx	0011	0011	0001	0000
xx	0010	0010		
xx	0001	0001	0000	
xx	0000	0000		

Intel x86汇编语言访问不同存储单元举例
（假设数据段寄存器DS的值为0）

```
MOV AL，[0x4]   # 将地址0x4开始的1字节存储单元的内容送入
                # 8位寄存器AL，结果为[AL]=0x00

MOV AX，[0x4]   # 将地址0x4开始的2字节存储单元的内容送入
                # 16位寄存器AX，结果为[AX]=0xEF00

MOV EAX，[0x4]  # 将地址0x4开始的4字节存储单元的内容送入
                # 32位寄存器EAX，结果为[EAX]=0xABCDEF00
```

从上述程序可以看出，不同的地址访问模式（字节访问、半字访问、字访问）所使用的主存地址实际上都是字节地址，CPU在执行指令的时候可以将字节地址的低2位用于访问控制：

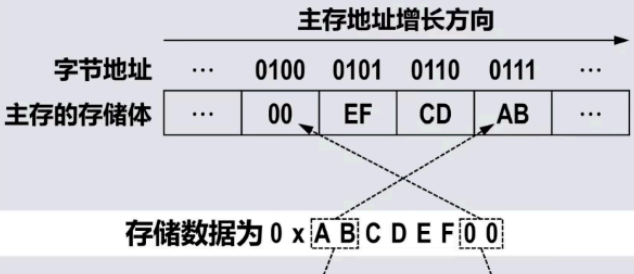
- 采用*字节访问模式，字节地址的低2位*用于选择字存储单元中的哪一个字节。
- 采用*半字访问模式，字节地址的倒数第2位*用于选择字存储单元中的哪个半字。

易错

- 字节地址右移的位数由“每字字节数”决定，不是固定 2 位
- 半字地址不是“字节地址右移 2 位”，而是右移 1 位（每半字 2 字节时）
- 地址对齐要求“起始地址是数据大小的整数倍”，不是“地址末位为 0”

三、大端和小端

主存中数据的存放 —— 大端和小端方式



小端方式

- 将数据的低字节保存在主存的低地址中，而数据的高字节保存在主存的高地址中。
- 这样可以将主存地址的高低与数据的位权有效地结合起来，高地址存储的数据部分的权值高，低地址存储的数据部分的权值低，符合逻辑。

大端方式

- 将数据的高字节保存在主存的低地址中，而数据的低字节保存在主存的高地址中。
- 符合人类的正常思维。

主存中数据的存放 —— 大端和小端方式

小端方式	大端方式
● 将数据的低字节保存在主存的低地址中，而数据的高字节保存在主存的高地址中。	● 将数据的高字节保存在主存的低地址中，而数据的低字节保存在主存的高地址中。
● 这样可以将主存地址的高低与数据的位权有效地结合起来，高地址存储的数据部分的权值高，低地址存储的数据部分的权值低，符合逻辑。	● 符合人类的正常思维。
● Intel x86、IA64、RISC-V等处理器采用小端方式。	● PowerPC处理器采用大端方式；ARM、MIPS等处理器同时支持大端方式和小端方式。

请同学们注意：

1. **上述两种方式并没有绝对的优劣之分，它们在不同的处理器架构和应用场景中都有各自的适用性和优势。**
2. **除处理器外，大小端方式还涉及外部设备设计、网络数据传输、音视频文件保存等。**
3. **小端与大端方式的区别不仅存在于处理器的寄存器、存储器中，在指令集、系统总线等各个层次中也可能存在差别。**

定义

- 大端（Big-Endian）：最高有效字节 MSB 放在最低地址
- 小端（Little-Endian）：最低有效字节 LSB 放在最低地址

例：32 位数据 `0x12345678` 存放在地址 `100` 开始的 4 个字节

地址	大端	小端
100	12	78
101	34	56
102	56	34
103	78	12

易错

- 大端/小端只影响“多字节数据的字节顺序”，不影响位序
- 大端不等于“高地址放高位”；要按“最低地址放 MSB/LSB”记忆
- 题目问“机器数”的存储顺序时，先确定数据宽度，再按字节排列
- 大小端与边界对齐是两个独立问题

四、数据的边界对齐

4.1 数据对齐的原因

主存中数据的存放 —— 数据的边界对齐

- **主存空间通常按字节进行编址。**
- **高级语言中不同数据类型的变量所包含的字节数量可能不同。**
  - 编译器在为这些变量分配主存空间时，理论上可以从主存空间的任何一个字节地址开始。
  - 当一个多字节变量被编译器分布在不同的字存储单元中时，访问该变量就需要多个存取周期。
  - 为了提高数据访问效率，应该要考虑数据变量、数据结构在主存空间中的边界对齐问题。

举例

主存中数据的存放 —— 数据的边界对齐

【举例】 假设存储字长为32位，为C语言不同数据类型的变量分配主存空间。

数据的边界*未对齐*

- 对存储空间的利用率最高
- 存在访问性能问题，例如：
  - 变量c的8个字节分布在3个存储单元中（访问该变量需要3个存取周期）
  - 变量e的2个字节分布在2个存储单元中（访问该变量需要2个存取周期）



数据的边界*对齐*

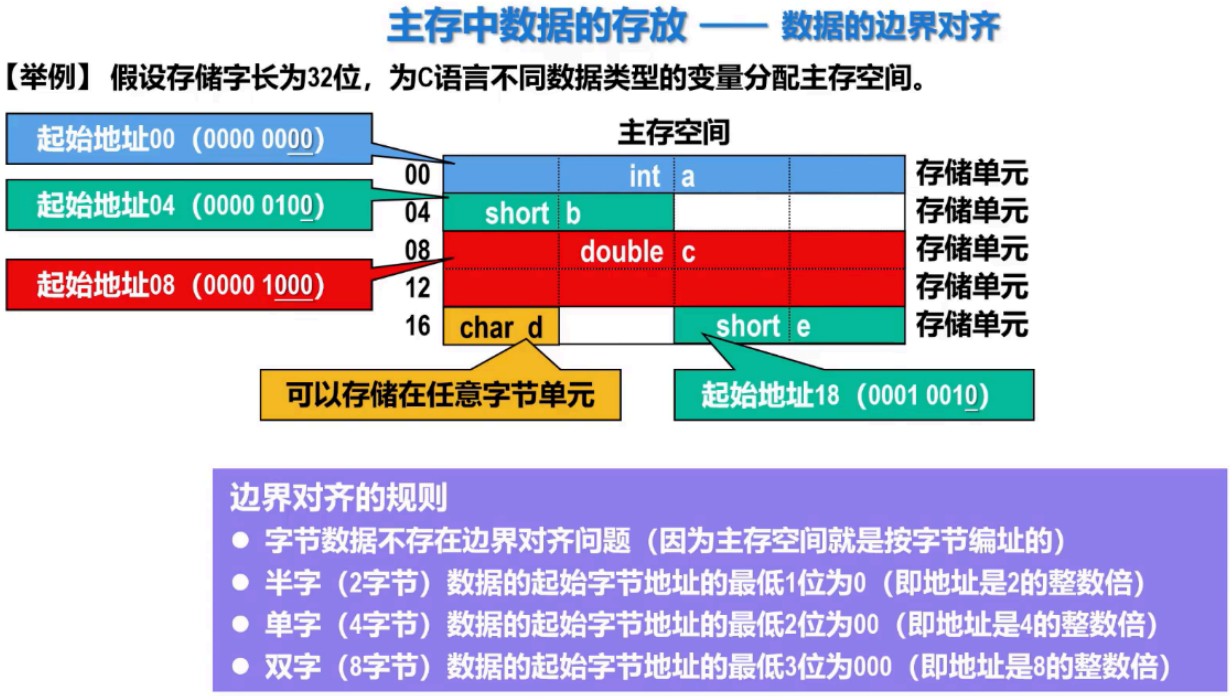
- 有效提升了访问性能，例如：
  - 变量c的8个字节分布在2个存储单元中（访问该变量需要2个存取周期）
  - 变量e的2个字节分布在1个存储单元中（访问该变量需要1个存取周期）
- 造成存储空间的浪费



提炼

- 对齐访问可以让一次总线传输完成；非对齐访问可能需要两次访存
- 结构体成员之间可能插入填充字节，导致 `sizeof(struct)` 大于成员大小之和

4.2 边界对齐的规则



规则

- 基本类型变量的起始地址必须是其大小的整数倍
- 结构体总大小必须是最大成员对齐数的整数倍
- 数组成员的起始地址按其元素大小对齐

例：`struct { char a; int b; short c; }`

成员	偏移	大小	填充
a	0	1	3
b	4	4	0
c	8	2	2
总大小	—	—	12

易错

- 对齐规则因编译器/架构而异，408 题目若未说明，按“自然对齐”处理
- 结构体总大小要补齐到最大对齐数的整数倍，不只是成员之间填充
- 大端/小端不影响结构体成员的偏移，只影响多字节成员的字节顺序

知识关联：存储字长（4.3 存储器的性能指标和存储系统层次结构）；主存结构（4.4 主存的结构）；指令字长与 PC 增量（5.1 指令的一般格式与指令字长）；数据表示（第 2 章）